What Makes a Good Plan for Effective Code Generation?

ANONYMOUS AUTHOR(S)

Planning has emerged as a foundational strategy in code intelligence. While many existing code generation approaches incorporate a planning module, the design of these plans is often heuristic, and there is limited empirical understanding of what constitutes a good plan for code generation. To address this gap, we conduct a systematic investigation of effective plans from three critical perspectives, grounded in extensive experiments on standalone function-level code generation benchmarks. First, we evaluate how different prompting strategies affect the quality of the generated plan and, subsequently, the performance and stability of the final code. Contrary to prevailing intuition, we find that more detailed self-planning does not improve results. Instead, plans elicited via relatively simple prompts, when grounded with visible test cases, achieve the best pass@1, test pass rate, and also generation stability. Second, we identify three key characteristics of effective plans: feasibility, requirement relevance, and clarity. Our analysis shows that feasibility is the strongest predictor of downstream code generation success, followed by relevance and clarity. Third, building on these findings, we propose PlanCoder, a quality-aware plan optimization framework. PlanCoder leverages visible tests to guide plan generation, repairs flawed plans before code synthesis, and employs plan-conditioned retrieval to support final code repair. Across two backbone models, PlanCoder improves average pass@1 over the strongest competing baseline by 4.4% and 5.0%, respectively, while requiring fewer LLM calls and lower token consumption.

CCS Concepts: • **Software and its engineering → Automatic programming**.

Additional Key Words and Phrases: Code generation, large language models, plan quality evaluation, program repair

1 Introduction

The remarkable progress in intelligent code synthesis has fundamentally reshaped modern software development, with planning emerging as a pivotal step in the generation workflow [Islam et al. 2024; Jiang et al. 2024; Li et al. 2023a; Wei et al. 2022; Yang et al. 2025b]. By decomposing a complex task into intermediate steps, planning provides explicit guidance and constraints that steer the model toward correct and coherent code [Jiang et al. 2024]. Despite its acknowledged importance, however, planning in current practice remains largely an exercise in heuristic prompt engineering: plans are generated by prompting LLMs, and rarely examined as an independent object of study [Islam et al. 2024; Ridnik et al. 2024]. This leaves a critical question squarely on the table: *how to make a good plan for effective code generation?*

Although there is no consensus on how to generate an effective plan, two prompt formats have become widely adopted for plan generation: self-planning [Jiang et al. 2024; Lei et al. 2025; Yang et al. 2025b] and what we term test-informed planning [Islam et al. 2024]. As shown in Fig. 1, the self-planning prompt adopts a structured format with pre-defined few-shot examples comprising k tasks and their corresponding plans, and explicitly instructs the planning agent to "Let's think step by step." In contrast, the test-informed planning prompt, exemplified by MapCoder, takes a simpler and more direct approach by providing concise instructions along with the visible test cases to guide the model toward solving the task. Despite the prevalence of these two formats, existing approaches typically adopt a single prompt format for plan generation without questioning its suitability. This motivates us to investigate **RQ1: which prompt format produces better plans for effective code generation?**

Uncertainty is an inherent characteristic of large language models (LLMs) [Chakraborti et al. 2025; Farquhar et al. 2024; Steyvers et al. 2025], meaning that even a well-designed prompt does

FSE '27, Shenzhen, China
2027.

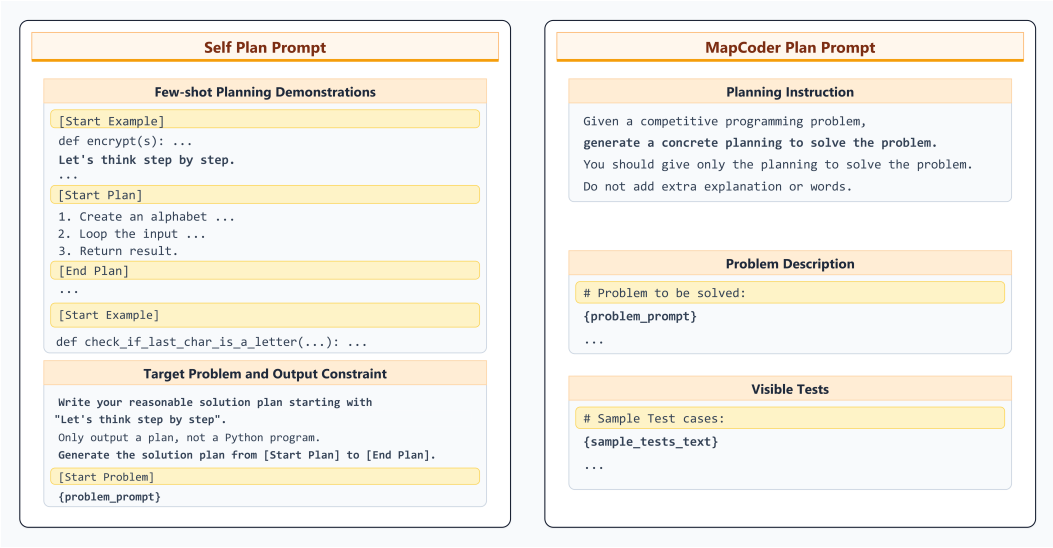


Fig. 1. Example prompts for self-planning (few-shot with structured examples) [Jiang et al. 2024] and test-informed planning (instructions with visible test cases) [Islam et al. 2024].

not guarantee a good plan in every generation. A common strategy to mitigate such uncertainty is to sample multiple outputs and select the most promising one [Besta et al. 2024; Hao et al. 2023; Jia et al. 2025; Mu et al. 2024; Yao et al. 2023]. However, this selection process critically depends on knowing what constitutes a good plan, yet little work has systematically characterized the properties of high-quality plans. Without such characterization, plan selection remains ad hoc and potentially suboptimal. This gap motivates our second research question: **RQ2: what are the characteristics of a good plan?**

Understanding the characteristics of a good plan benefits not only plan selection but also plan optimization. In practice, it is entirely possible that none of the candidate plans meets the quality bar; even the best among a set of candidates may still fall short. In such cases, selection alone is insufficient and optimization becomes essential. Recent work has shown that leveraging plan-level feedback can significantly improve code generation performance, and that reasoning over plans can offer more effective guidance than relying solely on code-level heuristics [Lei et al. 2025; Wang et al. 2025b]. These findings suggest that plan evaluation holds substantial potential for advancing generation quality. This leads to our third research question: **RQ3: to what extent can we advance code generation by leveraging feedback from plan evaluation?**

To address the first two research questions, we conduct empirical experiments on two widely used benchmarks, HumanEval-ET [Dong et al. 2025] and MBPP-ET [Dong et al. 2025]. For RQ1, we compare the two prompt formats in terms of the execution-based test pass rate and its variance across generated code samples. For RQ2, we analyze the characteristics of plans that lead to higher pass rates and lower variance, thereby identifying the distinguishing features of effective plans. For RQ3, we propose a novel code generation workflow, PlanCoder, and evaluate its performance against three state-of-the-art approaches on four benchmarks: MBPP-ET, APPS [Hendrycks et al. 2021], CodeContests [Li et al. 2022], and LiveCodeBench [Jain et al. 2024].

In RQ1, test-informed planning improves pass rate over direct generation by 4.48 and 13.40 percentage points on HumanEval-ET and MBPP-ET, respectively, while reducing pass-rate variance

by over 80% on both benchmarks and achieving the best overall performance among the evaluated strategies. In RQ2, feasibility has the highest Pearson correlation with test pass rate (0.293), followed by relevance (0.198) and clarity (0.102). In RQ3, PlanCoder improves average pass@1 over the strongest baseline by 4.4 and 5.0 percentage points with `gpt-5.4-mini` and `gpt-5.6-luna`, respectively; across benchmarks and models, it reduces LLM calls by 3.9–42.5% and wall-clock time by 9.4–59.2%, relative to the best baseline for each metric.

Our key findings are summarized as follows:

Finding I: Introducing plans consistently improves both the effectiveness and stability of generated code. Between the two formats, test-informed planning produces better plans, which in turn lead to higher pass rates and lower variance.

Finding II: A good plan exhibits three essential characteristics: feasibility (the plan logically solves the task and passes the provided tests), requirement relevance (the plan aligns with the problem specification), and clarity (the plan is unambiguous and well-structured). Among these, feasibility is the most critical.

Finding III: Our proposed PlanCoder consistently outperforms the completed baselines across all four benchmarks, improving average pass@1 by 4.4 and 5.0 percentage points with `gpt-5.4-mini` and `gpt-5.6-luna`, respectively. It also requires fewer LLM calls and less inference time on every benchmark.

2 Empirical Study: Addressing RQ1 and RQ2

2.1 RQ1: Better prompt for plan generation

2.1.1 Experimental Design. We conduct experiments using *DeepSeek-V4-Flash* as the backbone model on two executable code generation benchmarks: *HumanEval-ET* and *MBPP-ET*. Both benchmarks provide test suites that allow us to measure not only whether a generated program fully passes, but also the fraction of tests passed by partially correct programs.

To evaluate the effectiveness of planning strategies, we compare three generation settings: ① **No Plan**: Directly generate code from the problem description without any intermediate planning process. ② **Self-planning**: Use the prompt from Jiang et al. [Jiang et al. 2024] (see the left panel of Fig. 1). ③ **Test-informed Planning**: Use the prompt of MapCoder [Islam et al. 2024], a widely applied state-of-the-art method for function-level code generation that employs test-informed planning (see the right panel of Fig. 1).

To further examine how visible tests and prompt detail affect plan quality and downstream code quality, we add two configurations: ④ **Test-informed Self-planning**: Add the visible test cases to the original Self-planning prompt. ⑤ **MapCoder-style Planning without Test**: Remove the visible test cases from the original MapCoder-style prompt.

For each task and each setting, we generate five code samples. We report three metrics, all of which are averaged across tasks within each setting and benchmark. *Pass@1* counts a generated solution as correct only when its test pass rate is exactly 1.0, and averages this binary indicator. *Pass rate* measures the mean fraction of passed tests. $Variance_{P.R.}$ is first computed per task as the variance across its five generations, and then averaged across tasks. Higher pass@1 and pass rate indicate better functional correctness, while lower variance indicates more consistent generation quality.

2.1.2 Results. From Table 1, we can have two observations.

Observation 1: Explicit planning consistently improves both correctness and generation stability. Compared with direct code generation, planning-based strategies improve pass rates by 3.86–4.48% on HumanEval-ET and 1.20–13.40% on MBPP-ET, while reducing variance by over 80% on both benchmarks. The substantial variance reduction suggests that planning effectively narrows

Table 1. RQ1 results: Average performance (%) per benchmark across planning prompts.

Benchmark	Prompt Type	Pass@1	Pass Rate	$\text{Variance}_{\text{P.R.}}$
HumanEval-ET	No Plan	79.76%	91.66%	0.023
	Self-planning	80.37%	95.52%	0.003
	Test-informed Planning	**82.32%**	**96.14%**	**0.002**
	Test-informed Self-planning	81.46%	95.89%	0.0025
	MapCoder-style Planning without Test	79.88%	94.71%	0.0037
MBPP-ET	No Plan	50.43%	64.02%	0.023
	Self-planning	51.29%	65.22%	0.004
	Test-informed Planning	**60.76%**	**77.42%**	**0.003**
	Test-informed Self-planning	56.84%	72.36%	0.0034
	MapCoder-style Planning without Test	50.72%	64.58%	0.0052

the model's output distribution, making generation outcomes less sensitive to random decoding fluctuations. Notably, the improvement on MBPP-ET is considerably larger than on HumanEval-ET, likely because MBPP problems require more complex reasoning, where the structural guidance from planning offers greater relative benefit. Overall, these results indicate that explicit planning not only boosts correctness but, more importantly, enhances generation reliability by mitigating the inherent uncertainty of autoregressive decoding.

Observation 2: Test-informed planning (MapCoder-style) achieves the best overall performance. Compared with self-planning, test-informed planning consistently delivers stronger effectiveness and stability across both benchmarks. On HumanEval-ET, it improves pass@1 by 1.95% and pass rate by 0.62%, while further reducing variance by 33.3%. The gains are even more pronounced on MBPP-ET: pass@1 improves by 9.47%, pass rate by 12.20%, and variance decreases by 25.0%. The additional comparisons with test-informed self-planning and MapCoder-style planning without test show that the superiority of test-informed planning stems from two key design choices: the inclusion of public test cases that ground plan generation in concrete input–output behavior, and a concise prompt that minimizes reasoning overhead, allowing the model to focus on generating a correct plan that satisfies the tests. Together, these factors yield higher-quality plans, which directly translate into improved code correctness and reduced generation uncertainty.

2.2 RQ2: Characteristics of good plans

2.2.1 Plan Evaluation Metrics. To identify the critical characteristics that distinguish good plans from bad ones, we design three metrics, i.e., feasibility, relevance, and clarity, each capturing a distinct aspect of plan quality. For each metric, we compare the scores of good plans against those of bad plans. The underlying rationale is that a good plan should not only be logically sound and executable, but also closely aligned with the task requirements and easy to follow during implementation.

Feasibility measures whether a candidate plan contains sufficient and correct logic to solve the programming task and pass the provided test cases. Given the strong performance of LLM-based judges in code-related evaluation tasks [Pathak et al. 2025; Tseng et al. 2024], we employ an LLM to assess feasibility. The evaluation prompt includes the task description, the candidate plan, and the visible test cases. The model is asked to reason about whether the plan can be faithfully implemented to satisfy the task requirements and produce the expected outputs. Specifically, the LLM performs a lightweight mental simulation by analyzing: (1) whether the plan covers the key algorithmic steps



Fig. 2. Distribution of plan-quality scores for good and bad plans over 202 paired tasks. Boxes show the interquartile range, whiskers show the observed range, points show individual plans, and dark markers denote means.

required by the task; (2) whether important edge cases or constraints are omitted; and (3) whether the described logic is consistent with the expected test outputs. The LLM then outputs a feasibility score in the range of [0,1], with higher scores indicating greater likelihood of successful execution. This metric is essential because a plan that is infeasible, regardless of how well-written or relevant it is, cannot yield a correct implementation.

Relevance measures how well a candidate plan aligns with the specific requirements stated in the programming task. Using the same LLM-based judge, we prompt the model with the task description and the candidate plan, and ask it to determine whether the proposed plan genuinely targets the given problem. The evaluation considers four aspects: (1) whether the plan directly addresses the programming objective rather than describing unrelated procedures; (2) whether it covers the major functional requirements explicitly stated in the task description; (3) whether important information from the task description is omitted or only partially addressed; and (4) whether the overall focus remains consistent with the intended problem setting. The LLM outputs a relevance score in [0,1], where higher scores denote stronger alignment. This metric is important because a highly feasible plan is of little use if it solves a different problem than the one asked.

Clarity measures how understandable, concrete, and implementation-ready a candidate plan is. We prompt the LLM with the task description and the plan, and ask it to assess whether the plan is logically ordered, unambiguous, and sufficiently specific to be directly translated into code. The clarity score ranges from 0 to 1, with higher scores indicating greater ease of implementation. This metric captures the practical usability of a plan: even a feasible and relevant plan may fail to guide correct code generation if it is too vague or poorly structured to follow.

Together, these three metrics provide a comprehensive evaluation of plan quality from complementary perspectives, i.e., correctness, alignment, and usability, enabling us to systematically compare good and bad plans and uncover the key factors that contribute to effective planning.

2.2.2 Experimental Setup. We use the same backbone model and benchmarks as in Section 2.1. For each task, we adopt the better-performing prompt, namely the MapCoder prompt, to generate five plans. Each generated plan is then independently used to produce one code solution. We evaluate each generated code using two metrics: test pass rate (*Pass Rate*) and its variance across the five solutions for a given task ($Variance_{P.R.}$).

Since many tasks in these widely-used function-level benchmarks are too easy to reveal the benefits of planning, we filter for tasks that exhibit nontrivial variability in generation quality. Specifically, we select tasks with non-zero pass-rate variance. To facilitate meaningful plan analysis, we further require that at least one of the five generated solutions achieves a 100% pass rate. This filtering yields 202 valid tasks. For each task, we identify the *best plan* (the one that led to 100% pass rate) and the *worst plan* (the one that resulted in the lowest pass rate). When multiple plans are tied for the best or worst, we randomly choose one. This gives us 404 samples in total, each consisting of the task description, the best and worst plans, and their corresponding code implementations.

To identify the most significant characteristics of good plans, we compute the correlation between three plan quality metrics, and the *Pass Rate* of the generated code, using both *Pearson and Spearman* rank correlation coefficients. We additionally examine the distribution of each metric across the best and worst plans to better understand how these dimensions distinguish effective planning from ineffective ones.

2.2.3 Results. Table 2 shows that all three metrics are positively correlated with downstream test pass rate over all plans. *Feasibility* has the strongest association under both Pearson and Spearman coefficients, *Relevance* is the second strongest signal, while *Clarity* has the weakest association. Figure 2 further reveals that good plans exhibit upward shifts across all three metrics, with the clearest separation again appearing in feasibility, followed by relevance, and lastly clarity.

These findings suggest that a plan's logical soundness and its ability to satisfy task requirements are more critical to downstream code performance than surface-level readability. The relatively weak role of clarity can be attributed to the fact that large language models tend to generate plans with uniformly high clarity, and their strong comprehension capabilities further diminish the marginal benefit of clarity in determining success. In contrast, feasibility and relevance show greater variability across plans, making them more decisive factors in distinguishing effective plans from ineffective ones.

Table 2. Correlation between plan-quality metrics and downstream test pass rate over all plans.

Metric	Pearson	Spearman
Relevance	0.198	0.101
Clarity	0.102	0.079
Feasibility	**0.293**	**0.158**

2.2.4 Human–LLM Score Agreement. To assess whether LLM-based plan-quality scores agree with human judgment, we sampled 100 of the 202 tasks, stratified proportionally by benchmark, and retained both the best and worst plans for each task, yielding 200 plans. Three annotators with Python programming experience, none of whom were authors of this paper, independently scored feasibility, relevance, and clarity using the same criteria and $[0, 1]$ scale as the LLM judge. After calibration on separate examples, annotators evaluated plans in randomized order, without access to LLM scores, generated code, execution outcomes, or best/worst labels. They received the same task and plan inputs as the judge, including visible tests for feasibility.

For each plan i and dimension d, the human reference is the mean of the three ratings, $\bar{h}_{i,d} = \frac{1}{3}\sum_{r=1}^{3} h_{i,d,r}$. We used absolute-agreement ICC(A,3) [McGraw and Wong 1996] to assess the reliability of this mean. We compared LLM scores with human means using Lin's concordance correlation coefficient (CCC) [Lin 1989], Spearman's ρ, and mean absolute error (MAE). We computed 95% confidence intervals by bootstrapping tasks, retaining both plans from each sampled task.

Table 3. Human–LLM agreement for the three plan-quality dimensions. Parentheses report 95% confidence intervals.

Dimension	ICC(A,3) ↑	CCC ↑	Spearman ρ ↑	MAE ↓
Feasibility	0.909 (0.885–0.927)	0.898 (0.866–0.922)	0.903 (0.870–0.928)	0.052 (0.047–0.057)
Relevance	0.863 (0.821–0.894)	0.866 (0.818–0.901)	0.863 (0.813–0.899)	0.044 (0.040–0.049)
Clarity	0.853 (0.810–0.886)	0.839 (0.785–0.881)	0.836 (0.774–0.879)	0.034 (0.030–0.038)

Table 3 shows that the three-rater mean is reliable across dimensions, with ICC(A,3) ranging from 0.853 to 0.909. LLM scores agree closely with human means, yielding CCC values of 0.839–0.898, Spearman correlations of 0.836–0.903, and MAE of 0.034–0.052 on the $[0, 1]$ scale. The lower bounds of the 95% confidence intervals for CCC are at least 0.785. Feasibility exhibits the strongest concordance and rank agreement, whereas clarity has the smallest absolute error. These results support LLM-based assessment of feasibility, relevance, and clarity as approximations to human judgment within the evaluated sample.

3 Approach: PlanCoder

Based on the insights from RQ1 and RQ2, we propose PlanCoder, a simple but effective quality-aware workflow for plan-based code generation. As illustrated in Figure 3, the workflow operates in three phases. **Phase I** generates an initial plan and code using visible test-informed prompts. If the code fails the visible tests, **Phase II** initiates a plan-code repair loop. In each iteration, the plan is first repaired using evaluation feedback on feasibility, relevance, and clarity, with feasibility serving as the most distinctive signal according to RQ2; the repaired plan then guides the regeneration of the code, which is re-evaluated against the visible tests. If the number of plan-code repair iterations exceeds a predefined limit, **Phase III** switches to plan-conditioned code repair. In this final stage, the system retrieves plan-similar examples based on the final repaired plan and repairs the failing code until it passes the public tests or reaches the maximum allowed code-repair iterations.

If any generated implementation passes all public tests during the initial attempt, plan repair, or code repair, PlanCoder returns it immediately and evaluates it on the full hidden test suite. If no attempt passes public tests after exhausting both the plan-repair and code-repair iteration limits, PlanCoder returns the final repaired implementation.

3.1 Phase I: Test-Aware Plan and Code Generation

PlanCoder first prompts the model with the programming problem and its visible tests. The prompt requests only the concrete natural-language plan needed to solve the problem, avoiding extraneous explanation and overly detailed pseudocode [Yoo et al. 2024]. This approach mirrors the planning style of MapCoder while grounding the plan in public test behavior.

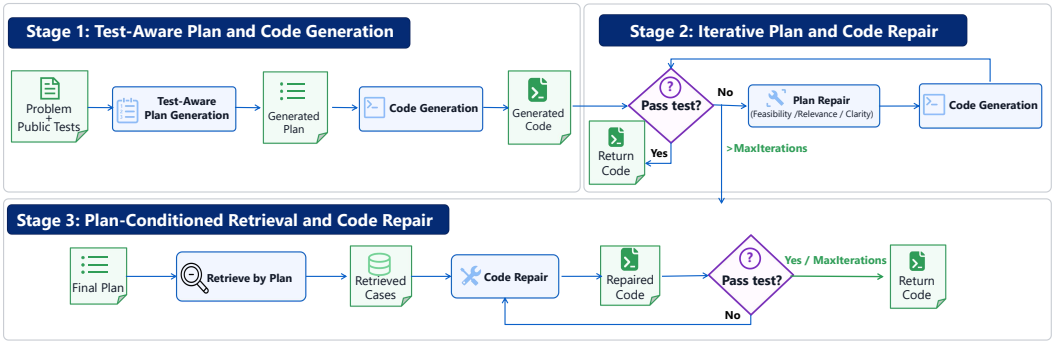


Fig. 3. Overview of the PlanCoder workflow.

3.2 Phase II: Iterative Plan and Code Repair

If the generated code fails any public test, PlanCoder constructs a repair prompt containing the problem description, the failed implementation, and the public tests. The prompt asks the model to produce a revised plan and explicitly addresses the three quality criteria identified in RQ2: relevance, feasibility, and clarity. Among these, feasibility is the primary focus, as RQ2 shows it to be the most distinctive characteristic of effective plans. In practice, the prompt specifically asks the model to explain how the revised plan would satisfy the provided tests. This loop continues until the generated code passes all public tests or the maximum number of plan-repair iterations is reached.

3.3 Phase III: Plan-Conditioned Retrieval and Code Repair

When plan-oriented repair in Phase II proves insufficient, PlanCoder transitions to iterative code-oriented repair, subject to a fixed maximum number of iterations. Before the first repair iteration, it uses the final repaired plan as the sole retrieval query, a novel design choice that distinguishes our approach from prior work. Unlike PERC [Yoo et al. 2024], we use a natural-language plan rather than pseudocode as the retrieval query. The retriever searches a plan-indexed pool of reference cases and reranks the retrieved cases by similarity to this final plan. The final repaired plan and the selected relevant cases then remain fixed throughout all subsequent code repair iterations.

At each iteration, the model receives the problem, public tests, the final repaired plan, the retrieved cases, the current failed code, and the failure information from the current iteration, and generates a revised implementation. The revised code is executed against the public tests. If it still fails and the maximum number of code-repair iterations has not been reached, the newly failed code and updated failure information are fed into the next iteration. This loop continues until the code passes all public tests or the maximum number of code-repair iterations is reached.

4 Evaluation: Addressing RQ3

We answer RQ3 by addressing the following three sub-RQs.

- **RQ3.1 (Effectiveness):** How effective is PlanCoder compared with iterative code-generation baselines?
- **RQ3.2 (Ablation Study):** How much do *iterative plan and code repair* (phase II) and *plan-conditioned retrieval and code repair*(phase III) contribute to the overall performance of PlanCoder?
- **RQ3.3 (Plan-Conditioned Retrieval):** Is retrieving reference cases by plans more effective than retrieving them by problem descriptions?

Table 4. Pass@1 for RQ3.1 across two backbone models (%). Within each model, bold = best, underline = strongest baseline, and subscripts = PlanCoder's improvement over that baseline.

Model	Method	MBPP-ET	APPS	CodeContests	LiveCodeBench	Avg.
`5.4-mini`	MapCoder	69.6	63.3	34.0	<u>78.6</u>	<u>61.4</u>
	DebateCoder	66.8	<u>64.7</u>	32.0	75.7	59.8
	LPW	<u>70.2</u>	64.0	<u>36.0</u>	72.9	60.8
	PlanCoder	**73.4**$_{\uparrow 3.2}$	**70.5**$_{\uparrow 5.8}$	**40.0**$_{\uparrow 4.0}$	**79.3**$_{\uparrow 0.7}$	**65.8**$_{\uparrow 4.4}$
`5.6-luna`	MapCoder	74.0	62.6	35.3	<u>82.1</u>	63.5
	DebateCoder	<u>75.2</u>	69.8	38.0	80.7	65.9
	LPW	73.2	<u>77.7</u>	<u>55.3</u>	81.4	<u>71.9</u>
	PlanCoder	**75.4**$_{\uparrow 0.2}$	**86.3**$_{\uparrow 8.6}$	**60.0**$_{\uparrow 4.7}$	**85.7**$_{\uparrow 3.6}$	**76.9**$_{\uparrow 5.0}$

4.1 Benchmarks and Models

We select four benchmarks that span a broad range of programming difficulty: MBPP-ET, APPS [Hendrycks et al. 2021], CodeContests [Li et al. 2022], and LiveCodeBench [Jain et al. 2024]. We evaluate all methods with two backbone models, `gpt-5.4-mini` and `gpt-5.6-luna`, to examine whether the results generalize across models.

4.2 Baselines

We compare against three state-of-the-art function-level code generation approaches. **MapCoder** [Islam et al. 2024] employs a multi-agent planning and code-refinement workflow. **DebateCoder** [Chen et al. 2025] improves solutions through test-driven debate among LLM agents, configured with four debate epochs. **LPW** [Lei et al. 2025] follows an iterative planning workflow with twelve iterations. We preserve the original configurations for all baselines.

All baselines and our approach are implemented using `gpt-5.4-mini` and `gpt-5.6-luna` as the underlying models. To ensure a fair comparison with MapCoder and LPW, which both employ twelve total generation attempts, we cap PlanCoder at the same budget: one initial generation (Phase I), up to five iterations in Phase II, and up to six iterations in Phase III.

The retrieval pool used in the approaches is built from the SWIFTCODER dataset [Huang et al. 2024], which comprises 9,451 programming problems with optimized Python solutions from eight open-source code datasets on Hugging Face. To prevent data leakage, we inspect SWIFTCODER's data sources and compare them with our evaluation benchmarks, identifying APPS as a shared source. We remove 21 identical problems appearing in the APPS test set from the retrieval pool and find no overlaps with the remaining benchmarks. This yields 9,430 problem–solution pairs. For plan-conditioned retrieval, we prompt DeepSeek-V4-Flash to generate an implementation plan for each pair based on the problem description and its solution code.

4.3 Metrics

In addition to the widely used **pass@1**, we adopt three complementary metrics to capture efficiency and overhead: **token consumption** as a proxy for inference cost, **LLM calls** as a measure of API invocation frequency, and **wall-clock time (s)** as an indicator of overall runtime overhead.

4.4 Addressing RQ3.1

Results with `gpt-5.4-mini`. Table 4 presents the pass@1 results, while Table 5 reports the efficiency metrics. As shown in Table 4, PlanCoder achieves the highest pass@1 across all four

Table 5. Inference cost for RQ3.1 across two backbone models. Within each model, bold marks PlanCoder when best, underline marks the strongest baseline, and subscripts show PlanCoder's reduction over that baseline.

Model	Method	MBPP-ET			APPS		
		Tok.	Calls	Time	Tok.	Calls	Time
`5.4-mini`	MapCoder	7507	9.24	44.83	24091	11.94	123.90
	DebateCoder	2483	4.82	21.06	12660	6.94	61.75
	LPW	27034	8.72	45.27	66281	15.37	94.64
	PlanCoder	**2019**$_{\downarrow 18.7}$	**3.40**$_{\downarrow 29.5}$	**14.41**$_{\downarrow 31.6}$	**10627**$_{\downarrow 16.1}$	**6.09**$_{\downarrow 12.2}$	**49.99**$_{\downarrow 19.0}$
`5.6-luna`	MapCoder	9138	8.25	83.92	26030	12.37	169.64
	DebateCoder	2716	3.77	44.41	9087	3.56	201.47
	LPW	19796	6.40	56.10	60540	12.02	129.95
	PlanCoder	**1269**$_{\downarrow 53.3}$	**2.17**$_{\downarrow 42.5}$	**18.13**$_{\downarrow 59.2}$	**5822**$_{\downarrow 35.9}$	**2.65**$_{\downarrow 25.7}$	**86.21**$_{\downarrow 33.7}$
Model	Method	CodeContests			LiveCodeBench		
		Tok.	Calls	Time	Tok.	Calls	Time
`5.4-mini`	MapCoder	30865	14.72	166.29	12584	9.29	81.91
	DebateCoder	19174	10.53	88.96	5021	4.40	31.84
	LPW	84187	18.97	134.40	38013	10.18	62.63
	PlanCoder	**15054**$_{\downarrow 21.5}$	**7.97**$_{\downarrow 24.3}$	**57.73**$_{\downarrow 35.1}$	5190	**4.23**$_{\downarrow 3.9}$	**22.78**$_{\downarrow 28.5}$
`5.6-luna`	MapCoder	30750	14.63	156.67	17548	8.86	181.97
	DebateCoder	12991	4.91	343.81	4660	3.04	68.40
	LPW	60493	11.91	148.71	26041	6.79	78.31
	PlanCoder	**7525**$_{\downarrow 42.1}$	**2.93**$_{\downarrow 40.3}$	**134.72**$_{\downarrow 9.4}$	**3479**$_{\downarrow 25.3}$	**2.58**$_{\downarrow 15.2}$	**40.61**$_{\downarrow 40.6}$

benchmarks, raising the average from the strongest baseline's 61.4% to 65.8%, a gain of 4.4 percentage points. The gains are especially clear on APPS and CodeContests, where PlanCoder improves over the strongest baseline by 5.8 and 4.0 points, respectively, indicating that the full workflow is effective on more challenging programming tasks.

Table 5 further shows that PlanCoder achieves these gains while also being more resource-efficient across nearly all metrics. It uses the fewest LLM calls and the lowest wall-clock time on all four benchmarks—most notably on CodeContests, where it reduces calls by 24.3% (from 10.53 to 7.97) and runtime by 35.1% (from 88.96s to 57.73s). On LiveCodeBench, DebateCoder consumes slightly fewer tokens; however, PlanCoder still delivers higher pass@1 with fewer calls and lower runtime.

Results with `gpt-5.6-luna`. PlanCoder again achieves the highest pass@1 on every benchmark. It improves the average from LPW's 71.9% to 76.9%, a gain of 5.0 percentage points based on the results. The gains over the strongest completed baseline are 0.2 points on MBPP-ET, 8.6 on APPS, 4.7 on CodeContests, and 3.6 on LiveCodeBench. The improvement is therefore not tied to `gpt-5.4-mini`: it persists with a second backbone.

The efficiency results show an equally consistent pattern. PlanCoder uses the fewest tokens, LLM calls, and wall-clock time on all four benchmarks. Relative to the second-best completed method, it reduces token consumption by 25.3–53.3%, calls by 15.2–42.5%, and runtime by 9.4–59.2%.

These savings are especially large on MBPP-ET, where PlanCoder reduces tokens by 53.3%, calls by 42.5%, and runtime by 59.2%. Together with the `gpt-5.4-mini` results, these findings show that PlanCoder's effectiveness and efficiency generalize across both evaluated backbones.

4.5 Addressing RQ3.2

Experimental setup. We conduct the ablation study with `gpt-5.4-mini` under the same benchmarks and evaluator used in RQ3.1. We compare three cumulative configurations. *Test-Aware Plan and Code Generation only* executes Phase I once without any repair. *+ Iterative Plan and Code Repair* additionally enables Phase II: whenever the implementation fails public tests, PlanCoder repairs the plan and regenerates code from the repaired plan for up to five iterations. *+ Plan-Conditioned Retrieval and Code Repair* is the full method, which further enables Phase III to retrieve reference cases using the final repaired plan and perform up to six code-repair iterations. All other settings are held fixed, so each row isolates the incremental contribution of the newly enabled phase.

Table 6. Ablation results for RQ3.2 using `gpt-5.4-mini` (pass@1, %). Each row cumulatively adds one repair stage; subscripts show the improvement over the preceding row.

Configuration	MBPP-ET	APPS	CodeContests	LiveCodeBench	Avg.
Phase I	66.4	53.2	34.0	73.6	56.8
+ Phase II	$70.2_{\uparrow 3.8}$	$64.7_{\uparrow 11.5}$	$38.0_{\uparrow 4.0}$	$77.9_{\uparrow 4.3}$	$62.7_{\uparrow 5.9}$
+ Phase III (full)	$\mathbf{73.4}_{\uparrow 3.2}$	$\mathbf{70.5}_{\uparrow 5.8}$	$\mathbf{40.0}_{\uparrow 2.0}$	$\mathbf{79.3}_{\uparrow 1.4}$	$\mathbf{65.8}_{\uparrow 3.1}$

Results. Table 6 shows that both repair phases make consistent and complementary contributions. Adding *Iterative Plan and Code Repair* (Phase II) improves average pass@1 from 56.8% to 62.7% (+5.9%). The gains are 3.8% on MBPP-ET, 11.5% on APPS, 4.0% on CodeContests, and 4.3% on LiveCodeBench. APPS exhibits the largest gain. The remaining three benchmarks show gains within the narrower range of 3.8–4.3%, so the average improvement is not driven solely by APPS.

The subsequent *Plan-Conditioned Retrieval and Code Repair* (Phase III) further raises average pass@1 from 62.7% to 65.8% (+3.1%). It adds 3.2% on MBPP-ET, 5.8% on APPS, 2.0% on CodeContests, and 1.4% on LiveCodeBench. As with Phase II, APPS receives the largest improvement. Phase III provides a positive but smaller increment than Phase II on every benchmark, consistent with their respective average gains of 3.1% and 5.9%.

Overall, the full workflow outperforms Phase I (classical plan usage) by 9.0% on average. The cumulative gains are 7.0% on MBPP-ET (66.4% to 73.4%), 17.3% on APPS (53.2% to 70.5%), 6.0% on CodeContests (34.0% to 40.0%), and 5.7% on LiveCodeBench (73.6% to 79.3%). Phase II contributes the bulk of the improvement (+5.9% average), while Phase III delivers additional gains once plan-level issues have been addressed.

4.6 Addressing RQ3.3

Experimental setup. We compare two retrieval-query representations using `gpt-5.4-mini` on the same four benchmarks as RQ3.1. Both variants employ identical PlanCoder settings, differing only in the retrieval query: *problem-description* and *the repaired plan. Problem-description retrieval*, the most common setting in current research [Wang et al. 2025a; Yang et al. 2025b; Zhou et al. 2022], indexes and queries cases by their original problem descriptions. In contrast, *plan-conditioned retrieval* indexes cases by their plans and uses the final repaired plan as the query. The retrieved problem–code pairs are then fed into the same downstream code-repair prompt.

Table 7. RQ3.3 results using `gpt-5.4-mini`.

Retrieval query	MBPP-ET				APPS			
	Pass@1	Tok.	Calls	Time	Pass@1	Tok.	Calls	Time
Problem des.	70.8	**1849**	**3.37**	15.61	67.6	13397	6.68	51.93
Plan des.	**73.4**$_{\uparrow 2.6}$	2019$_{\uparrow 9.2}$	3.40$_{\uparrow 1.0}$	**14.41**$_{\downarrow 7.7}$	**70.5**$_{\uparrow 2.9}$	**10627**$_{\downarrow 20.7}$	**6.09**$_{\downarrow 8.7}$	**49.99**$_{\downarrow 3.7}$
Retrieval query	CodeContests				LiveCodeBench			
	Pass@1	Tok.	Calls	Time	Pass@1	Tok.	Calls	Time
Problem des.	37.3	16371	8.13	67.30	78.6	**4622**	**3.85**	26.75
Plan des.	**40.0**$_{\uparrow 2.7}$	**15054**$_{\downarrow 8.0}$	**7.97**$_{\downarrow 2.0}$	**57.73**$_{\downarrow 14.2}$	**79.3**$_{\uparrow 0.7}$	5190$_{\uparrow 12.3}$	4.23$_{\uparrow 9.8}$	**22.78**$_{\downarrow 14.8}$

Results. As shown in Table 7, using the repaired plan as the retrieval query improves pass@1 on every benchmark. Pass@1 increases from 70.8% to 73.4% on MBPP-ET (+2.6%), from 67.6% to 70.5% on APPS (+2.9%), from 37.3% to 40.0% on CodeContests (+2.7%), and from 78.6% to 79.3% on LiveCodeBench (+0.7%). The gains are similar on the first three benchmarks and smaller on LiveCodeBench, increasing the macro average from 63.6% to 65.8% (+2.2%).

Across the 929 paired tasks, the two retrieval variants produce different correctness outcomes on 60 tasks. Plan-conditioned retrieval solves 41 tasks that problem-description retrieval misses, whereas the reverse occurs for 19 tasks, yielding a net gain of 22 solved tasks. An exact McNemar test on these discordant pairs gives $p = 0.006$, indicating that the improvement is statistically significant.

At the macro-average level, plan-conditioned retrieval reduces token consumption by 9.2%, LLM calls by 1.5%, and wall-clock time by 10.3%. On APPS and CodeContests, all three cost metrics decrease alongside the gains in pass@1. For example, token consumption on APPS falls from 13,397 to 10,627, while wall-clock time on CodeContests decreases from 67.30s to 57.73s. On MBPP-ET and LiveCodeBench, tokens and LLM calls increase, but wall-clock time still decreases from 15.61s to 14.41s and from 26.75s to 22.78s, respectively. Thus, runtime is lower on all four benchmarks, while token and call savings vary by benchmark. These results support the use of the repaired plan as the retrieval query: unlike the surface form of a problem description, the repaired plan exposes the intended algorithmic strategy and incorporates information gained during plan repair, making the retrieved cases more useful for resolving the remaining implementation errors.

5 Discussion

5.1 Implications

Our empirical findings emphasize the importance of high-level planning for code quality. RQ1 shows that explicit planning consistently improves code correctness and generation stability compared with direct code generation. Among the evaluated planning strategies, test-informed planning achieves the best overall performance on both benchmarks.

RQ2 provides a complementary basis for directing refinement. Feasibility has the strongest association with downstream correctness among the three evaluated dimensions, suggesting that plan revision should first examine whether the proposed logic can solve the task. Relevance and clarity remain useful for checking requirement coverage and making the solution actionable. These criteria provide a structured way to inspect plans, while execution feedback determines whether the resulting implementation passes the available tests.

PlanCoder incorporates these insights into a staged workflow. Plan repair provides an opportunity to revise the algorithmic strategy before further implementation changes, while subsequent code repair addresses failures that remain. Using the repaired plan for retrieval also connects the intended solution strategy to external examples. Together, these findings motivate treating plans as intermediate artifacts that can be evaluated, revised, and reused throughout generation. Extending this role to project-level development would require maintaining consistency across modules and dependencies.

5.2 Limitations and Opportunities

First, our plan-quality metrics are model-judged. Although the judges follow explicit rubrics with zero-temperature scoring, they may still inherit biases from the underlying model. The human–LLM agreement analysis supports the use of these scores within the evaluated sample, but agreement with human ratings does not establish that every highly rated plan is correct. Broader human assessment across models and task types would strengthen this evidence. Future work could also combine rubric-based judgments with static analysis or symbolic execution of corresponding implementations to check whether the proposed reasoning is reflected in executable behavior.

Second, public test suites are inherently incomplete. A plan refined to pass public tests may overfit to visible examples while failing on hidden edge cases. We reserve the hidden tests for final evaluation, separating the feedback used during refinement from the assessment of the returned implementation. Nevertheless, passing either suite does not establish correctness for all possible inputs. Future work could investigate test selection and augmentation that expose missing boundary conditions or branches, while assessing whether the resulting gains justify the additional generation and execution cost.

Third, our current PlanCoder evaluation is limited to standalone programming benchmarks. Repository-level or project-level code generation introduces additional planning challenges, such as API constraints, dependency management, and project-specific conventions. A locally feasible plan may still conflict with another component or rely on an incorrect assumption about an existing interface. Extending our approach would therefore require evaluating plans against repository context and revisiting dependent plans when requirements or implementations change. Whether the present quality dimensions and repair schedule remain effective in such settings requires further study.

5.3 Threats to Validity

Internal validity. In RQ1 and RQ2, we sample five generations per task and setting and aggregate execution outcomes to reduce sensitivity to stochastic variation. This procedure supports comparisons under the evaluated sampling conditions, but cannot eliminate variability or establish significance for every observed difference. In RQ3, methods are compared using the same backbone models and benchmark evaluators, with generation limits specified in the experimental setup. The results characterize the evaluated workflows; the cumulative ablation does not isolate every possible interaction between their components.

Construct validity. Pass@1 and test pass rate measure correctness against the benchmark tests, while pass-rate variance measures consistency across sampled implementations. These measures do not capture all aspects of software quality or model uncertainty. Similarly, feasibility, relevance, and clarity are rubric-based assessments of plans. Human agreement supports their interpretability, but the observed correlations with execution outcomes are modest and do not establish a causal effect of any individual dimension.

External validity. Our findings are based on standalone Python benchmarks and the selected backbone models. The RQ2 analysis additionally focuses on tasks with variable outcomes and at

least one successful generation, enabling comparisons between effective and ineffective plans for the same task. This selection may underrepresent tasks that models consistently solve or consistently fail. The relative value of concise planning and staged repair may also change with the language, task domain, model capability, or available tests. Broader evaluations are needed before generalizing these findings to other settings.

6 Related Work

6.1 Planning and Reasoning for Code

Chain-of-thought prompting elicits explicit intermediate reasoning [Wei et al. 2022]. For code generation, structured chain-of-thought organizes this reasoning around sequence, branch, and loop structures [Li et al. 2023b]. Self-planning explicitly separates planning from implementation: the model first decomposes the task into solution steps and then generates code guided by those steps [Jiang et al. 2024]. These approaches establish intermediate plans as an interface between natural-language requirements and executable programs.

Subsequent work incorporates planning into more elaborate generation workflows. MapCoder coordinates agents for recalling examples, planning, code generation, and debugging [Islam et al. 2024]. PlanSearch explores diverse natural-language observations and plans to increase the diversity of candidate solutions [Wang et al. 2025b]. Our study complements these workflow and search approaches by examining the plans themselves. We investigate how visible tests and prompting strategies affect downstream correctness and stability, and analyze feasibility, requirement relevance, and clarity as characteristics of effective plans.

6.2 Iterative Repair and Multi-Agent Generation

Feedback-driven refinement improves generated programs through repeated evaluation and revision. Reflexion converts task feedback into verbal reflections that inform subsequent attempts [Shinn et al. 2023]. AlphaCodium organizes code generation as a test-based, multi-stage iterative process [Ridnik et al. 2024]. DebateCoder uses test cases and execution outcomes to support debate between models, allowing them to challenge and refine each other's implementations [Chen et al. 2025]. These methods demonstrate different ways of incorporating feedback into code generation.

LPW is particularly relevant because it verifies solution plans against visible tests before implementation and uses the resulting verification to guide subsequent code refinement [Lei et al. 2025]. It therefore already establishes a role for plan-level reasoning in iterative programming. PlanCoder builds on this direction through a workflow informed by our empirical analysis of plan quality. Its repair prompts explicitly address feasibility, relevance, and clarity, with particular emphasis on satisfying visible tests. When plan repair reaches its iteration limit, the workflow uses the final repaired plan to retrieve examples for further code repair.

6.3 Quality Assessment for Code Generation

Existing research investigates how to assess generated programs beyond textual similarity. CodeScore learns execution-related properties to estimate functional correctness [Dong et al. 2025]. Rubric-based evaluation uses problem-specific criteria to assess the logic of student programs [Pathak et al. 2025], while CodEv combines LLM-based grading, ensembles, and agreement checks to improve consistency with human evaluation [Tseng et al. 2024]. These studies motivate the use of explicit assessment criteria and validation against human judgments.

Our evaluation focuses on intermediate natural-language plans. We define three dimensions—feasibility, requirement relevance, and clarity—and examine their associations with downstream test pass rate. We additionally compare LLM scores with the mean ratings of three human annotators.

This connects plan assessment to both human judgment and execution outcomes, providing an empirical basis for the criteria used in PlanCoder's refinement process.

6.4 Retrieval-Augmented Code Generation

Retrieval augments code generation with external information relevant to the target task. Parvez et al. [2021] retrieve code and summaries to support generation, while DocPrompting retrieves documentation conditioned on natural-language intent [Zhou et al. 2022]. Empirical studies further examine how the quality and integration of retrieved code affect generation performance, highlighting the importance of selecting useful context [Yang et al. 2025a].

PERC is closely related because it already uses algorithmic plans for example retrieval. Its pseudocode representation reduces syntactic noise and supports retrieval within and across programming languages [Yoo et al. 2024]. PlanCoder uses a natural-language plan as the retrieval query. Moreover, this query is the final plan produced by the preceding repair stage, and the retrieved examples support subsequent code repair. Our retrieval experiment compares this representation with problem-description retrieval under the same workflow settings, assessing its effect on both correctness and inference cost.

7 Conclusion

What defines a good plan for code generation? Through systematic experimentation on function-level coding tasks, we demonstrate that simple MapCoder-style plans, grounded in limited visible test cases, provide the most favorable balance of correctness and stability. Our analysis identifies feasibility, requirement relevance, and clarity as core attributes of effective plans, with feasibility standing out as the primary driver of code quality and robustness. Guided by these findings, we develop PlanCoder, a framework that incorporates public-test-guided plan generation, proactive repair of flawed plans, and plan-based retrieval for code repair. On four diverse benchmarks, MBPP-ET, APPS, CodeContests, and LiveCodeBench, PlanCoder not only achieves higher pass@1 but also reduces inference time and resource consumption compared to strong iterative baselines. We hope this granular investigation sheds light on the role of planning in code generation and inspires future efforts along this promising avenue.

8 Data Availability

The benchmark datasets are available from the sources cited in this paper. Our code and data supplement is available for anonymous review at https://figshare.com/s/732460e69017557aafeb.

References

Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, et al. 2024. Graph of thoughts: Solving elaborate problems with large language models. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 38. 17682–17690.

Tapabrata Chakraborti, Christopher R. S. Banerji, Ariane Marandon, Vicky Hellon, Robin Mitra, Brieuc Lehmann, Leandra Bräuninger, Sarah McGough, Cagatay Turkay, Alejandro F. Frangi, Ginestra Bianconi, Weizi Li, Owen Rackham, Deepak Parashar, Chris Harbron, and Ben MacArthur. 2025. Personalized uncertainty quantification in artificial intelligence. *Nature Machine Intelligence* 7, 4 (2025), 522–530. doi:10.1038/s42256-025-01024-8

Jizheng Chen, Kounianhua Du, Xinyi Dai, Weiming Zhang, Xihuai Wang, Yasheng Wang, Ruiming Tang, Weinan Zhang, and Yong Yu. 2025. DebateCoder: Towards Collective Intelligence of LLMs via Test Case Driven LLM Debate for Code Generation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Vienna, Austria, 12055–12065. doi:10.18653/v1/2025.acl-long.589

Yihong Dong, Jiazheng Ding, Xue Jiang, Ge Li, Zhuo Li, and Zhi Jin. 2025. Codescore: Evaluating code generation by learning code execution. *ACM Transactions on Software Engineering and Methodology* 34, 3 (2025), 1–22.

Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, and Yarin Gal. 2024. Detecting hallucinations in large language models using semantic entropy. *Nature* 630, 8017 (2024), 625–630. doi:10.1038/s41586-024-07421-0

Shibo Hao, Yi Gu, Haodi Ma, Joshua Hong, Zhen Wang, Daisy Wang, and Zhiting Hu. 2023. Reasoning with language model is planning with world model. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 8154–8173.

Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, et al. 2021. Measuring coding challenge competence with apps. *arXiv preprint arXiv:2105.09938* (2021).

Dong Huang, Guangtao Zeng, Jianbo Dai, Meng Luo, Han Weng, Yuhao Qing, Heming Cui, Zhijiang Guo, and Jie M Zhang. 2024. Effi-code: Unleashing code efficiency in language models. *arXiv preprint arXiv:2410.10209* (2024).

Md Ashraful Islam, Mohammed Eunus Ali, and Md Rizwan Parvez. 2024. MapCoder: Multi-Agent Code Generation for Competitive Problem Solving. *arXiv preprint arXiv:2405.11403* (2024).

Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2024. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974* (2024).

Haoxiang Jia, Robbie Morris, He Ye, Federica Sarro, and Sergey Mechtaev. 2025. Automated Repair of Ambiguous Problem Descriptions for LLM-Based Code Generation. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)* (Seoul, Korea, Republic of). IEEE Press, 367–379. doi:10.1109/ASE63991.2025.00038

Xue Jiang, Yihong Dong, Lecheng Wang, Zheng Fang, Qiwei Shang, Ge Li, Zhi Jin, and Wenpin Jiao. 2024. Self-planning code generation with large language models. *ACM Transactions on Software Engineering and Methodology* 33, 7 (2024), 1–30.

Chao Lei, Yanchuan Chang, Nir Lipovetzky, and Krista A. Ehinger. 2025. Planning-Driven Programming: A Large Language Model Programming Workflow. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Vienna, Austria, 12647–12684. doi:10.18653/v1/2025.acl-long.621

Jia Li, Ge Li, Yongmin Li, and Zhi Jin. 2023b. Structured chain-of-thought prompting for code generation. *ACM Transactions on Software Engineering and Methodology* (2023).

Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. 2023a. StarCoder: may the source be with you! *arXiv preprint arXiv:2305.06161* (2023).

Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. 2022. Competition-level code generation with alphacode. *Science* 378, 6624 (2022), 1092–1097.

Lawrence I-Kuei Lin. 1989. A Concordance Correlation Coefficient to Evaluate Reproducibility. *Biometrics* 45, 1 (1989), 255–268. doi:10.2307/2532051

Kenneth O. McGraw and S. P. Wong. 1996. Forming Inferences About Some Intraclass Correlation Coefficients. *Psychological Methods* 1, 1 (1996), 30–46. doi:10.1037/1082-989X.1.1.30

Fangwen Mu, Lin Shi, Song Wang, Zhuohao Yu, Binquan Zhang, ChenXue Wang, Shichao Liu, and Qing Wang. 2024. ClarifyGPT: A Framework for Enhancing LLM-Based Code Generation via Requirements Clarification. *Proc. ACM Softw. Eng.* 1, FSE, Article 103 (July 2024), 23 pages. doi:10.1145/3660810

Md Rizwan Parvez, Wasi Uddin Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. 2021. Retrieval augmented code generation and summarization. *arXiv preprint arXiv:2108.11601* (2021).

Aditya Pathak, Rachit Gandhi, Vaibhav Uttam, Arnav Ramamoorthy, Pratyush Ghosh, Aaryan Raj Jindal, Shreyash Verma, Aditya Mittal, Aashna Ased, Chirag Khatri, et al. 2025. Rubric is all you need: Improving llm-based code evaluation with question-specific rubrics. In *Proceedings of the 2025 ACM Conference on International Computing Education Research V. 1*. 181–195.

Tal Ridnik, Dedy Kredo, and Itamar Friedman. 2024. Code generation with alphacodium: From prompt engineering to flow engineering. *arXiv preprint arXiv:2401.08500* (2024).

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems* 36 (2023), 8634–8652.

Mark Steyvers, Heliodoro Tejeda, Aakriti Kumar, Catarina Belem, Sheer Karny, Xinyue Hu, Lukas W. Mayer, and Padhraic Smyth. 2025. What large language models know and what people think they know. *Nature Machine Intelligence* 7, 2 (2025), 221–231. doi:10.1038/s42256-024-00976-7

En-Qi Tseng, Pei-Cing Huang, Chan Hsu, Peng-Yi Wu, Chan-Tung Ku, and Yihuang Kang. 2024. Codev: An automated grading framework leveraging large language models for consistent and constructive feedback. In *2024 IEEE International Conference on Big Data (BigData)*. IEEE, 5442–5449.

Evan Z Wang, Federico Cassano, Catherine Wu, Yunfeng Bai, William Song, Vaskar Nath, Ziwen Han, Sean M. Hendryx, Summer Yue, and Hugh Zhang. 2025b. Planning in Natural Language Improves LLM Search for Code Generation. In *International Conference on Learning Representations*. https://mlanthology.org/iclr/2025/wang2025iclr-planning/

Zora Zhiruo Wang, Akari Asai, Xinyan Velocity Yu, Frank F. Xu, Yiqing Xie, Graham Neubig, and Daniel Fried. 2025a. CodeRAG-Bench: Can Retrieval Augment Code Generation?. In *Findings of the Association for Computational Linguistics: NAACL 2025*, Luis Chiruzzo, Alan Ritter, and Lu Wang (Eds.). Association for Computational Linguistics, Albuquerque, New Mexico, 3199–3214. https://aclanthology.org/2025.findings-naacl.176/

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.

Donghao Yang, Aolang Wu, Tianyi Zhang, Li Zhang, Xiaoli Lian, Fang Liu, Yuming Ren, Jiaji Tian, and Xiaoyin Che. 2025b. AutoPLC: Generating Vendor-Aware Structured Text for Programmable Logic Controllers. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 3593–3604. doi:10.1109/ASE63991.2025.00297

Zezhou Yang, Sirong Chen, Cuiyun Gao, Zhenhao Li, Xing Hu, Kui Liu, and Xin Xia. 2025a. An Empirical Study of Retrieval-Augmented Code Generation: Challenges and Opportunities. *ACM Transactions on Software Engineering and Methodology* (2025).

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems* 36 (2023), 11809–11822.

Jaeseok Yoo, Hojae Han, Youngwon Lee, Jaejin Kim, and Seung-won Hwang. 2024. PERC: Plan-As-Query Example Retrieval for Underrepresented Code Generation. *arXiv preprint arXiv:2412.12447* (2024).

Shuyan Zhou, Uri Alon, Frank F Xu, Zhiruo Wang, Zhengbao Jiang, and Graham Neubig. 2022. Docprompting: Generating code by retrieving the docs. *arXiv preprint arXiv:2207.05987* (2022).